FlowShift: Targeted and Reversible Traffic Transformation for Encrypted Flow Privacy

Sourabh Singh, Kunvar Kanhaiya, Pralhad Magadum, and Aniruddha Singh Kushwaha
Department of Computer Science & Engineering Indian Institute of Technology Indore, India
(msrphd2304101012, phd2201101015, phd2201201007, phd2401101006, aniruddha)@iiti.ac.in

***Abstract*—Encrypted network traffic exposes behavioral characteristics (e.g. packet sizes, timing, etc.). These metadata patterns can be exploited by machine-learning-based classifiers to identify the underlying service even when the packet contents remain inaccessible. Existing defenses are largely based on one-way traffic transformations and predominantly rely on packet-size manipulation. Such approaches can impose substantial bandwidth overhead while offering limited flexibility in reshaping traffic characteristics. To mitigate the issues properly, we introduce *FlowShift*, a cooperative and reversible traffic-transformation framework that decouples the statistical representation of a flow from its original service. FlowShift jointly modifies packet-size, timing, and volume characteristics while preserving the original application data. We evaluate FlowShift against state-of-the-art classifiers using a diverse set of encrypted traffic traces. The results demonstrate that FlowShift effectively reduces the accuracy of service classification by up to 85% while accommodating moderate bandwidth and FCTs overhead, thereby enhancing communication privacy. Furthermore, we validate FlowShift in a real-time Linux-based emulation environment with practical file-transfer workloads, confirming its feasibility in live network scenarios.**

***Index Terms*—Encrypted Traffic Analysis, Metadata Privacy, Flow Fingerprint, Privacy Preserving Communications, Traffic Obfuscation.**

I. Introduction

The widespread adoption of end-to-end encryption has significantly strengthened communication privacy across modern networks. Protocols such as TLS 1.3 [], QUIC [], HTTPS, and Virtual Private Network (VPN) tunneling [] prevent intermediate entities from directly accessing application-layer payloads, thereby substantially limiting traditional payload-based inspection mechanisms such as Deep Packet Inspection (DPI). However, encryption primarily protects the *content* of communication and does not conceal the observable characteristics generated during transmission. Packet sizes, directions, inter-arrival times (IATs), packet counts, flow duration, traffic volume, and burst structures remain visible to an intermediate observer and can form distinctive statistical fingerprints of the underlying application. Earlier studies demonstrated that such metadata can be exploited to infer encrypted communication activities without recovering plaintext content []. Consequently, the increasing use of encryption has shifted traffic analysis from payload inspection toward statistical inference over network-visible metadata.

This capability has become considerably stronger with recent advances in machine learning. Early encrypted-traffic classifiers primarily relied on manually engineered flow statistics, such as packet-size distributions, inter-arrival times, flow duration, packet counts, and byte volumes, which were subsequently processed using conventional models including Random Forests [], Support Vector Machines [], and ensemble methods []. More recent approaches have shifted toward deep learning, enabling classifiers to learn discriminative representations directly from packet sequences and flow traces without requiring access to encrypted payloads.

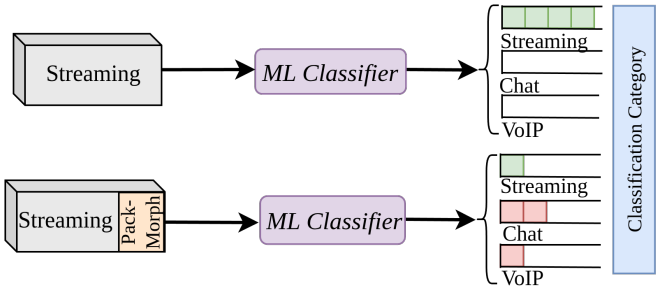


Fig. 1. Flow classification using ML classifier (Top) and introducing flow misclassification with FlowShift (Bottom).

Figure 1 contrasts conventional ML-based identification of an encrypted flow with FlowShift's objective of reshaping its observable features so that the same classifier associates the transformed flow with a plausible target service.

Deep Packet [], for example, demonstrated effective encrypted application identification using deep neural architectures, while Deep Fingerprinting [] showed that convolutional models can capture subtle structural patterns from encrypted traffic traces. Similarly, k-fingerprinting [] demonstrated that compact statistical representations can retain sufficient information for reliable traffic identification. Importantly, subsequent studies have shown that classification does not depend on a single observable characteristic; rather, packet size, direction, timing, flow volume, burst structure, and temporal ordering provide complementary information that can jointly reveal the underlying service. As a result, suppressing or modifying only one feature may still leave sufficient residual information for a classifier to recover the original traffic identity. For instance, timing-based approaches have demonstrated that inter-packet timing alone can provide strong discriminative information [], while robust fingerprinting methods have shown that identifiable traffic patterns may persist even after defensive transformations are applied [].

Although encrypted-traffic classification supports legitimate applications such as Quality-of-Service provisioning, intrusion detection, traffic engineering, resource management, and network monitoring [], the same capability introduces a significant privacy concern. A passive observer can potentially infer whether a user is streaming multimedia, transferring files, participating in a VoIP session, accessing remote services, or interacting with specific applications or websites without decrypting the packet contents or compromising the underlying cryptographic protocol.

To mitigate metadata leakage, several traffic-analysis defenses have been proposed. Early approaches primarily rely on packet padding, dummy-packet insertion, and traffic tunneling to obscure observable characteristics of encrypted flows []. While these mechanisms can reduce the information exposed through individual features, they often modify only a limited portion of the traffic representation and may therefore leave other discriminative characteristics unchanged. For example, normalizing packet sizes may conceal size-related patterns while preserving distinctive properties such as packet counts, and burst structure. More sophisticated defenses consequently attempt to reshape larger portions of the traffic trace. Walkie-Talkie [] modifies burst patterns to reduce trace distinguishability, FRONT [] introduces randomized dummy traffic, RegulaTor [] applies structured traffic shaping, and Mockingbird [] uses adversarial transformations to mislead learning-based fingerprinting models. More recent approaches have also explored transport-assisted and programmable-network mechanisms, including QCSD [] and Minos [], to provide stronger control over observable traffic characteristics. Despite these advances, existing defenses continue to face an important trade-off between privacy and communication efficiency. Aggressive transformations typically require substantial padding, dummy packets, or additional traffic volume, resulting in increased bandwidth consumption and communication overhead, whereas lightweight transformations may preserve sufficient residual structure for advanced classifiers to identify the original service.

More importantly, many existing defenses perturb traffic without explicitly controlling the resulting observable behavior. Such transformations may leave residual or artificial patterns that remain exploitable by advanced classifiers. This motivates a more structured approach that jointly reshapes multiple traffic characteristics while limiting communication overhead. To address this gap, we introduce *FlowShift*, a cooperative and reversible traffic-transformation framework that reduces service-level distinguishability while preserving end-to-end communication correctness. The main contributions of this work are summarized as follows:

1) **Cooperative and Reversible Traffic Transformation:** We design *FlowShift*, an end-to-end framework that modifies the observable characteristics of encrypted traffic while preserving the original application data. In contrast to unilateral traffic obfuscation, FlowShift coordinates the transformation across communication endpoints to support reversible traffic reshaping and correct data recovery.
2) **Structured Multi-Dimensional Traffic Reshaping:** We develop a structured transformation strategy that jointly alters multiple observable traffic characteristics rather than relying on isolated or purely random perturbations. This design reduces the persistence of service-specific traffic patterns across multiple statistical dimensions while controlling the communication overhead introduced by the transformation.
3) **Evaluation Against Service Classification:** We evaluate *FlowShift* using VPN and non-VPN traffic from the MIT-VNAT dataset against machine-learning-based service classifiers. The results show a substantial reduction in service-classification accuracy, demonstrating the effectiveness of FlowShift in limiting metadata-based service inference.
4) **End-to-End System Validation:** We implement *FlowShift* in a Virtual Machine based emulation environment and evaluate it using practical file-transfer workloads. The experiments confirm bit-exact recovery of the original data while quantifying the bandwidth and flow-completion-time overhead introduced by the transformation.

The remainder of this paper is organized as follows. Section II reviews related work on encrypted-traffic analysis and traffic obfuscation. Section III presents the FlowShift architecture and transformation methodology. Section IV describes the experimental setup and evaluation results. Section V discusses performance trade-offs and practical considerations. Finally, Section VI concludes the paper.

II. Preliminaries

In this section, we first describe the threat model considered in this work and then identify the key requirements that an effective encrypted-traffic obfuscator should satisfy. These requirements are derived from the limitations of existing traffic-analysis defenses and from the practical constraints associated with deploying traffic transformation in an end-to-end communication setting. Finally, we motivate the need for a new solution by highlighting why existing approaches do not simultaneously satisfy all of these requirements.

A. Threat Model

Our threat model considers two trusted endpoints communicating over an encrypted connection, with a passive adversary positioned along the communication path. Similar to prior encrypted-traffic analysis studies [], the adversary cannot decrypt packet payloads, compromise the communicating endpoints, modify the application data. The adversary can observe network-visible information such as packet sizes, packet directions, packet counts, flow volume, burst behavior, and the evolution of these characteristics during the lifetime of a flow. By collecting these observations, the adversary can construct a statistical representation of the communication and use it to infer the application or service associated with the encrypted flow.

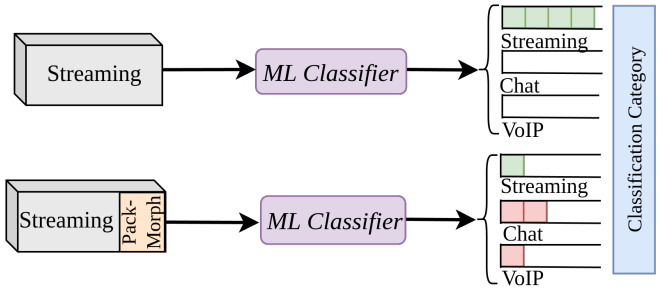


Fig. 2. Threat model for encrypted-traffic analysis. A passive adversary observes the traffic characteristics without decrypting the payloads.

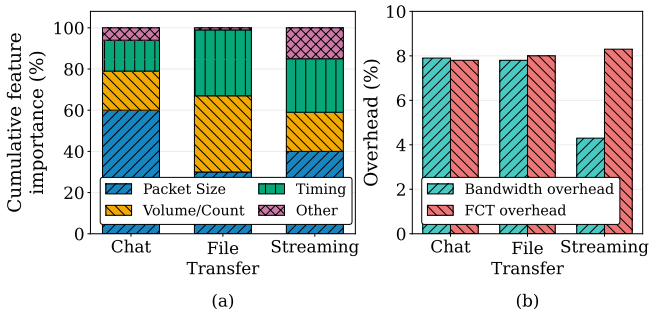


Fig. 3. VPN traffic analysis for Chat, File Transfer, and Streaming using the VNAT dataset []: (a) cumulative importance of packet-size, volume/count, timing, and other feature groups; and (b) bandwidth and flow-completion-time (FCT) overhead introduced by *Dolos*.

TABLE I
COMPARISON WITH REPRESENTATIVE TRAFFIC-ANALYSIS DEFENSES.

Defense	Multi-Feature Obfuscation	Adaptive Transformation	Endpoint Coordination	Low BW/FCT Overhead
FRONT []	?	?	?	?
RegulaTor []	?	?	?	?
Mockingbird []	?	?	?	?
Dolos []	?	?	?	?
FlowShift	?	?	?	?

We assume a strong adversary capable of applying statistical, machine-learning, or deep-learning classifiers to the observed traffic []. The attacker is not restricted to a particular classifier architecture and may exploit multiple traffic characteristics jointly when performing service inference. We additionally assume that the adversary may be aware that traffic obfuscation is being applied and may retrain its classifier using similarly transformed traffic. Therefore, the effectiveness of the defense should not depend on hiding the transformation mechanism or on access to a particular attacker's model. In contrast, the communicating endpoints are trusted and may participate in the obfuscation process. The goal of the defender is therefore to reduce service-level information leakage from observable traffic metadata while preserving the correctness and semantics of the original end-to-end application communication.

B. Key Requirements

Based on the above threat model, we identify five key requirements for an effective traffic obfuscator: *multi-feature obfuscation*, *adaptive transformation*, *endpoint coordination*, and *low bandwidth/FCT overhead*. These requirements capture both the privacy objective of reducing service distinguishability and the practical objective of maintaining usable network performance. As summarized in Table I, representative traffic-analysis defenses satisfy different subsets of these properties, but none simultaneously provides all of them.

Multi-Feature Obfuscation: Encrypted traffic exposes several correlated characteristics that can jointly reveal the identity of the underlying service. Modern classifiers have demonstrated that packet sizes, directions, packet counts, flow volume, and burst-related behavior can all contribute useful discriminative information []. Consequently, modifying only one observable characteristic may not be sufficient to prevent inference. For example, changing packet sizes can reduce size-based information while leaving packet counts, directional patterns, or flow volume largely unchanged. An adaptive classifier may then rely more heavily on these remaining characteristics and continue to recover the original service identity. Several existing defenses primarily focus on a limited subset of the available side-channel information [], which can leave residual patterns in other dimensions. An effective obfuscator should therefore modify multiple observable characteristics jointly so that the protection does not depend on suppressing only a single traffic feature. As shown in Fig. 3(a), packet size, volume/count, and timing jointly account for more than 75% of the cumulative feature importance across the evaluated VPN services in the VNAT dataset []. These metadata groups therefore provide most of the discriminative information used to train the classifier, reinforcing the need to obfuscate them jointly.

Adaptive Transformation: A fixed transformation that applies the same modification to every flow can itself create a stable and learnable metadata pattern. This limitation is especially important when the adversary knows that a defense is active and can collect examples of its output []. The adversary can retrain its classifier on the resulting packet-size, timing, volume, and burst characteristics, thereby adapting the model to the transformed traffic behavior. Protection may consequently degrade once the retrained model learns these new regularities and associates them with the original service. Moreover, applying a uniform modification ignores natural differences in flow size, duration, and application behavior, potentially providing weak protection for some flows while imposing excessive overhead on others. An effective obfuscator should therefore adapt the type and degree of transformation to each flow and vary its behavior over time []. Such adaptation prevents a single deterministic modification from becoming a reliable fingerprint while allowing the defense to balance privacy protection against communication overhead.

Endpoint Coordination: Traffic transformation becomes

increasingly difficult as the number of modified traffic characteristics increases. Simple unilateral defenses can introduce padding or additional packets without changing the original application stream, but more extensive transformations may alter packet boundaries, introduce artificial data, or reorganize how the original information is transmitted []. Without coordination, the receiving side may be unable to distinguish transformed traffic from genuine application data or correctly reconstruct the original communication. This limits how aggressively a one-sided defense can reshape observable traffic while maintaining correctness. Coordination between trusted endpoints provides a mechanism for applying richer transformations at the sender while enabling the receiver to correctly interpret and recover the original application stream. Such cooperation can therefore support stronger traffic obfuscation without requiring modifications to the application itself. For this reason, endpoint coordination is an important requirement for maintaining communication correctness under multi-feature traffic transformation.

Low Bandwidth/FCT Overhead: Traffic obfuscation inevitably introduces additional communication cost. Many existing defenses achieve stronger protection by inserting dummy packets, padding real packets, regulating transmissions, or buffering data before transmission []. Although these mechanisms can reduce the amount of useful information available to an observer, they may also significantly increase the number of transmitted bytes or extend the time required to complete a flow []. High bandwidth overhead can reduce effective network utilization and cause artificial traffic to compete with legitimate application data. Similarly, excessive buffering or reshaping can increase Flow Completion Time (FCT), directly affecting user-perceived performance and application responsiveness. These costs become particularly important for large file transfers, bandwidth-constrained links, or environments where multiple protected flows coexist. An effective obfuscator should therefore balance privacy improvement with communication efficiency and keep both bandwidth and FCT overhead within practical limits. Fig. 3(b) reports both overheads for the evaluated VPN services, illustrating the need to minimize these costs while still providing effective protection.

The Need for a New Solution: As shown in Table I, existing traffic-analysis defenses provide valuable protection but generally satisfy only a subset of the requirements identified above. Padding- and traffic-shaping mechanisms can modify several observable characteristics, but stronger transformations often introduce considerable bandwidth or latency overhead []. Adaptive and adversarial techniques provide more targeted changes, yet they may rely on particular traffic patterns, classifier behavior, or attack assumptions []. Other approaches introduce endpoint-side mechanisms but do not jointly combine adaptive multi-feature obfuscation, coordinated communication, low bandwidth/FCT overhead, and complete end-to-end validation []. Consequently, there remains a need for a framework that treats these requirements as complementary rather than independent design objectives. This motivates *FlowShift*, which is designed to provide coordinated and adaptive traffic transformation while preserving application correctness and maintaining practical communication overhead.

III. FlowShift Overview

The architecture of *FlowShift*, shown in Fig. 4, implements the coordinated transformation and application-data preservation requirements introduced in Section II. FlowShift operates as an optional shim between the application and transport layers. The sender-side manager intercepts outgoing application traffic and reshapes it before forwarding it to the transport layer; the receiver-side manager restores the original application stream.

The architecture separates policy selection, packet-level execution, and receiver-side restoration. This section identifies the components responsible for these tasks. Section IV then describes how a selected policy is executed and reversed. With those operations established, Section V defines how profiles and communication costs guide policy selection.

A. Sender-Side Components

The sender-side manager turns a flow's transformation requirements into an executable packet plan through two components: the Transformation Policy Selector and the Transformation Planner.

Transformation Policy Selector (TPS): At flow establishment, the TPS uses the associated source profile, available flow information, and configured displacement and overhead requirements to evaluate candidate policies. It first excludes candidates that violate the flow's constraints, then retains the non-dominated candidates that offer efficient displacement–cost trade-offs. The output is the selected policy set $\mathcal{P}_f$, which is passed to the planner. The TPS operates on policy specifications and cost estimates; profile construction and the selection rule are developed in Section V.

Transformation Planner (TP): The TP receives $\mathcal{P}_f$ and converts its policies into packet-level actions. For transformation interval k, it uses the flow's seeded sequence to obtain an active policy $p_k \in \mathcal{P}_f$ and a holding time Δt_k. It then realizes the policy's size, volume, and timing requirements using fragmentation, padding, artificial traffic, and buffering. The operations are described in Section IV, and the shared sequence is defined in Section IV-D.

These actions change the observable traffic while retaining the application bytes. The receiver must therefore obtain both the policy configuration and the restoration information needed to interpret the transformed packets.

B. Receiver-Side Components

Configuration Manager (CM): At flow establishment, the receiver obtains the seed ξ, the ordered list Π_f of policies in $\mathcal{P}_f$, and the common initial synchronization state. The CM uses this configuration to reproduce the sender's policy–duration sequence and activate the restoration configuration M_k for each interval.

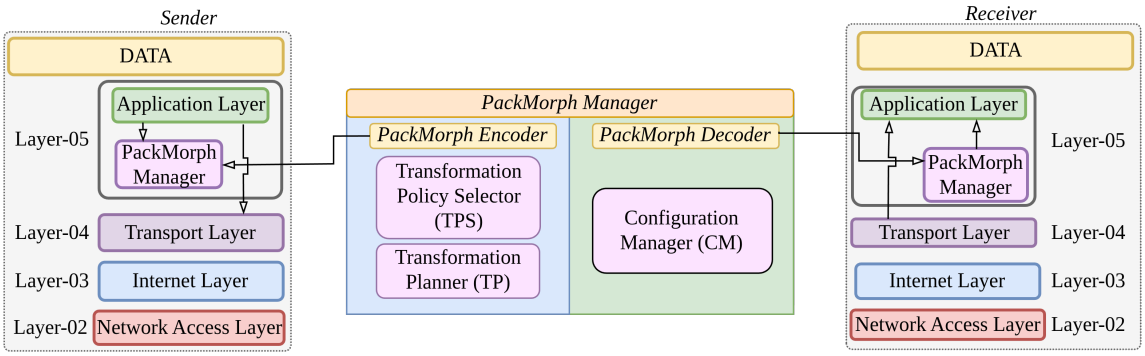


Fig. 4. Overall architecture and component interaction of *FlowShift*.

Using that configuration and the protected restoration metadata, the receiver removes artificial traffic and padding, reassembles fragments, and delivers the recovered bytes to the destination application in their original order. Configuration identifies the applicable inverse operations, while packet metadata supplies the valid lengths and fragment information needed to carry them out. Section IV-D specifies the relationship between the shared schedule and restoration state.

Together, the sender and receiver provide the execution path for a selected policy. The next section follows that path from packet-size and volume adjustment through release timing to coordinated restoration.

IV. FlowShift Operations

The components in Section III execute a policy through three coupled traffic transformations. For this section, assume that the TPS has supplied the selected set $\mathcal{P}_f$ and that the TP has obtained policy p_k for an interval of duration Δt_k. The policy is represented as

$$p_k = (S_k, T_k, V_k), \tag{1}$$

where S_k specifies packet-size behavior, T_k specifies packet-release timing, and V_k specifies traffic volume. The following subsections explain how the TP forms transformed packets, adjusts their aggregate volume, and schedules their release. The final subsection establishes how both endpoints follow the same policy sequence and restore the application stream. Policy selection itself is addressed in Section V.

A. Packet-Size Transformation

The packet-size operation changes the network-visible size of outgoing packets without altering the application bytes. Let L_i be the payload length of application packet i, and let $\widehat{L}_i$ be the target size determined by S_k. FlowShift performs one of the following actions:

$$\mathcal{O}_{\text{size}}(L_i, \widehat{L}_i) = \begin{cases} \text{fragment}, & L_i > \widehat{L}_i, \\ \text{pad}, & L_i < \widehat{L}_i, \\ \text{forward}, & L_i = \widehat{L}_i. \end{cases} \tag{2}$$

For fragmentation, the payload is divided into ordered pieces whose valid bytes jointly equal the original payload. For padding, artificial bytes are appended until the target size is reached. FlowShift records the valid payload length and fragment information required by the receiver to remove padding and reconstruct the original packet. Fragmentation and padding also change packet count and byte volume, so their output supplies the input to the volume adjustment below.

B. Traffic-Volume Transformation

Given the packets produced by size transformation, the traffic-volume operation controls the network-visible packet count during interval k. Let N_k^{target} be the count required by V_k, and let N_k^{real} be the number of transformed application packets available during that interval. The number of artificial packets is

$$N_k^{\text{dummy}} = \max\left(0, N_k^{\text{target}} - N_k^{\text{real}}\right). \tag{3}$$

Artificial packets carry no application data and are marked in the protected restoration information so that the receiver can discard them. If the available application traffic exceeds the target for the current interval, the TP buffers the excess data for a later release opportunity; genuine application data are never discarded. In this way, FlowShift changes the observable packet count and byte volume while preserving stream correctness. The release timing of these genuine and artificial packets is governed jointly by T_k, as described next.

C. Timing Transformation

Once packet sizes and the intended volume are specified, the timing operation determines when packets are released. Here, Δt_k denotes the duration for which policy p_k remains active, whereas T_k defines the packet-level timing behavior within that interval. In particular, the TP represents the target inter-packet delays as

$$T_k = \left[\delta_{k,1}, \delta_{k,2}, \ldots, \delta_{k,N_k^{\text{target}}}\right], \tag{4}$$

where $\delta_{k,j}$ is the target delay between the $(j-1)^{\text{th}}$ and j^{th} transmitted packets. Let $a_{k,j}$ denote the time at which packet j becomes available to the TP and let $r_{k,j}$ denote its release time. Taking $r_{k,0} = t_k$, the start of interval k, makes $\delta_{k,1}$ the initial release offset. The release schedule is computed as

$$r_{k,j} = \max(a_{k,j}, r_{k,j-1} + \delta_{k,j})\,. \tag{5}$$

If a packet becomes available before its scheduled release time, the TP temporarily buffers it; otherwise, the packet is released as soon as it becomes available. This operation modifies the observable timing pattern while preserving packet order. Artificial packets used to fill release opportunities are accounted for by the volume requirement V_k. The TPS evaluates the estimated buffering cost against the FCT-overhead constraint in Section V; realized release times also depend on packet availability. These size, volume, and timing changes require the receiver to identify the applicable policy and preserve restoration state across intervals.

D. Synchronization and Restoration

The preceding operations describe execution under a given p_k. Synchronization supplies that policy consistently at both endpoints, and restoration recovers the bytes carried by the transformed traffic.

Shared policy sequence: At flow establishment, the endpoints obtain the flow seed ξ and a deterministic ordering $\Pi_f = (p_1^{(f)}, \ldots, p_R^{(f)})$ of the selected policy set $\mathcal{P}_f$, where $R = |\mathcal{P}_f| > 0$. The construction of this set is described in Section V. Both endpoints use the same pseudorandom generator and holding-time bounds, and retain the same ordered policy list throughout the flow.

For interval index $k \geq 0$, the Transformation Planner (TP) generates the active policy and its duration as

$$(u_k, v_k) = \mathrm{PRG}(\xi, k), \tag{6}$$

$$i_k = 1 + (u_k \bmod R), \qquad p_k = p_{i_k}^{(f)}, \tag{7}$$

$$\Delta t_k = \Delta t_{\min} + v_k(\Delta t_{\max} - \Delta t_{\min}), \tag{8}$$

where $u_k \in \mathbb{Z}_{\geq 0}$, $v_k \in [0, 1)$, and $0 < \Delta t_{\min} \leq \Delta t_{\max}$. Starting at t_0, the sender applies p_k over $[t_k, t_{k+1})$, with $t_{k+1} = t_k + \Delta t_k$. The holding time specifies how long the policy remains active; its timing component T_k specifies packet releases within that interval. Consecutive selections may repeat, and $R = 1$ gives a fixed policy.

The receiver reproduces $(p_k, \Delta t_k)$ from the shared generator inputs and derives the corresponding restoration configuration M_k. This avoids signaling a new policy identifier at every transition. Reproducing the sequence requires agreement on the interval index and boundaries; the seed alone does not identify the interval of a delayed packet or compensate for clock drift.

Restoration of application data: For traffic associated with interval k, the receiver applies the inverse operations specified by p_k, together with the protected restoration metadata and reassembly state. It discards artificial packets, removes padding using the valid payload lengths, and reassembles fragmented application data. The restoration process is represented as

$$\mathbf{X}_k = \mathcal{R}\Big(\widehat{\mathbf{X}}_k, p_k\Big)\,, \tag{9}$$

where $\widehat{\mathbf{X}}_k$ denotes the transformed traffic associated with interval k and $\mathbf{X}_k$ the recovered application-data sequence. The metadata and receiver state are implicit inputs to $\mathcal{R}$. Recovered bytes are delivered in their original order once the required fragments are available. Restoration preserves the application stream rather than reconstructing its original transmission times. Advancing the policy sequence and retaining the state needed for outstanding data together support restoration across policy transitions.

The execution mechanism establishes which transformations can preserve application data and where bandwidth and delay costs arise. The next section uses these properties to construct candidate policies and select efficient transformations for each flow.

V. FlowShift Optimization Strategy

The operations in Section IV provide the means to reshape a flow; the TPS determines which combinations should be used. FlowShift aims to maximize statistical displacement while minimizing bandwidth and delay costs, with delay measured by flow-completion-time (FCT) overhead. We formulate this trade-off as a constrained multi-objective problem. The first subsection constructs profiles and candidate policies offline. The second evaluates those candidates for an individual flow and returns the selected set $\mathcal{P}_f$ used by the architecture in Section III.

A. Traffic Profile and Candidate Policy Construction

The offline stage translates representative service behavior into requirements for the size, timing, and volume operations already defined.

Profile estimation: Let $\mathcal{C} = \{c_1, \ldots, c_K\}$ denote the service classes and $\mathcal{X}_c = \{x_{c,1}, \ldots, x_{c,n_c}\}$ the representative traces of class c. For each feature group $g \in \{\mathrm{size}, \mathrm{time}, \mathrm{vol}\}$, an extractor Φ_g obtains trace-level descriptors and an estimator $\mathcal{E}_g$ summarizes them across the class:

$$\boldsymbol{\mu}_{c,g} = \mathcal{E}_g(\{\Phi_g(x) : x \in \mathcal{X}_c\})\,, \tag{10}$$

$$\boldsymbol{\mu}_c = [\boldsymbol{\mu}_{c,\mathrm{size}}, \boldsymbol{\mu}_{c,\mathrm{time}}, \boldsymbol{\mu}_{c,\mathrm{vol}}], \tag{11}$$

$$\mathcal{L} = \{\boldsymbol{\mu}_c : c \in \mathcal{C}\}. \tag{12}$$

The components describe packet lengths, inter-arrival times, and packet counts or byte volumes, respectively. Each component can contain distribution parameters or several summary statistics; it need not be a single mean. Feature definitions, scaling, and observation intervals are fixed across classes so that profiles are comparable. The library therefore represents class-level behavior, while individual flows may exhibit variation around that behavior. All profile estimation is performed offline.

Candidate construction: For a source class c_s and a distinct target class c_t, a policy is represented as

$$p_{s,t,a} = \mathcal{G}(\boldsymbol{\mu}_{c_s}, \boldsymbol{\mu}_{c_t}; a) = (S_{s,t,a}, T_{s,t,a}, V_{s,t,a}), \tag{13}$$

where $a \in \mathcal{A}_{s,t}$ indexes configured transformation variants. The mapping $\mathcal{G}$ converts the source–target comparison into packet-size requirements S, release-timing requirements T, and traffic-volume requirements V. Variants may use different amounts of padding, buffering, or artificial traffic for the same target. The candidate bank is

$$\mathcal{B} = \bigcup_{c_s \in \mathcal{C}} \bigcup_{c_t \in \mathcal{C} \setminus \{c_s\}} \{p_{s,t,a} : a \in \mathcal{A}_{s,t}\}. \tag{14}$$

Each entry is associated with its source and target classes and estimated communication costs. A profile supplies the statistical reference; a policy specifies shaping requirements; the TP converts those requirements into the packet operations described in Section IV. The three requirements must be considered jointly: fragmentation changes packet count, padding increases byte volume, and buffering changes both release times and the traffic visible within an interval.

Candidate realizability follows the application-byte restoration requirement in Section IV-D. Writing $\mathcal{T}_p$ for forward transformation, $\mathcal{R}_p$ for restoration using the required metadata, and $\mathrm{bytes}(x)$ for the ordered application stream, this condition is

$$\mathrm{bytes}(\mathcal{R}_p(\mathcal{T}_p(x))) = \mathrm{bytes}(x). \tag{15}$$

The transformation may change packet boundaries, release times, and artificial traffic, while preserving the application stream. Its communication costs must also satisfy the flow-specific constraints defined below. Thus, the offline output is a bank of source–target policies whose suitability must still be assessed for each new flow.

B. Multi-Objective Policy Selection

Given the candidate bank $\mathcal{B}$, the TPS evaluates which policies suit flow f. It first restricts the bank to compatible, realizable candidates, enforces the flow's constraints, and then retains the feasible candidates offering non-dominated trade-offs between displacement, bandwidth, and delay. This produces the selected set $\mathcal{P}_f$ passed to the TP.

Displacement and communication costs: For flow f with source-class association $c(f)$, let $\mathcal{B}_f \subseteq \mathcal{B}$ contain the policies constructed for that class that can preserve the flow's application bytes. Let $\mathbf{z}_f$ be the baseline traffic-feature representation and $\widehat{\mathbf{z}}_f(p)$ its estimated representation under policy p. Define

$$D_f(p) = d(\widehat{\mathbf{z}}_f(p), \mathbf{z}_f)\,, \tag{16}$$

where d uses consistent feature scaling. A larger value indicates greater departure from the source behavior; classifier evasion and target-class imitation are evaluated separately. For positive baseline byte count b_f and FCT τ_f, the relative costs are

$$O_{\mathrm{BW}}(f,p) = \frac{\widehat{b}_f(p) - b_f}{b_f}, \tag{17}$$

$$O_{\mathrm{FCT}}(f,p) = \frac{\widehat{\tau}_f(p) - \tau_f}{\tau_f}, \tag{18}$$

where $\widehat{b}_f(p)$ and $\widehat{\tau}_f(p)$ are the transformed byte count and FCT estimates. At flow establishment, the TPS estimates baseline and transformed quantities from the source profile and available flow information under the same accounting conventions. Size and volume operations contribute to transmitted-byte cost, while buffering and release scheduling contribute to FCT cost. These quantities connect the packet operations to the optimization objectives below.

Constrained optimization: With candidate policy p as the decision variable, the problem is

$$\begin{aligned} \underset{p \in \mathcal{B}_f}{\text{maximize}} \quad & \begin{bmatrix} D_f(p) \\ -O_{\mathrm{BW}}(f,p) \\ -O_{\mathrm{FCT}}(f,p) \end{bmatrix} \\ \text{subject to} \quad & D_f(p) \geq D_f^{\min}, \\ & O_{\mathrm{BW}}(f,p) \leq B_f^{\max}, \\ & O_{\mathrm{FCT}}(f,p) \leq F_f^{\max}. \end{aligned} \tag{19}$$

The negative cost terms express bandwidth and delay minimization within the vector maximization. The constraints impose a minimum displacement $D_f^{\min}$ and maximum bandwidth and FCT overheads $B_f^{\max}$ and $F_f^{\max}$. Let $\mathcal{F}_f$ denote the subset of $\mathcal{B}_f$ satisfying all three inequalities. Feasibility determines which policies are acceptable; optimization compares their displacement–cost trade-offs.

Selection of non-dominated policies: The objectives may conflict because increasing displacement can require more padding or buffering. We therefore interpret the optimization in the Pareto sense. A feasible policy q dominates a feasible policy p, written $q \succ_f p$, if

$$\begin{aligned} D_f(q) &\geq D_f(p), \\ O_{\mathrm{BW}}(f,q) &\leq O_{\mathrm{BW}}(f,p), \\ O_{\mathrm{FCT}}(f,q) &\leq O_{\mathrm{FCT}}(f,p), \end{aligned} \tag{20}$$

with at least one strict inequality. Thus, q improves at least one objective without worsening the others, making p unnecessary under these objectives. The TPS retains

$$\mathcal{P}_f = \{p \in \mathcal{F}_f : \nexists q \in \mathcal{F}_f \text{ with } q \succ_f p\}. \tag{21}$$

This set contains the Pareto-optimal candidates: improving one objective requires sacrificing another within the available feasible bank. The TPS obtains it by rejecting constraint violations and then removing dominated policies. This allows multiple efficient trade-offs without requiring fixed weights for the three objectives; distinct policies with identical objective values can both remain.

If $\mathcal{F}_f = \varnothing$, no policy satisfies the flow's requirements. Otherwise, the TPS passes $\mathcal{P}_f$ to the TP for the coordinated execution described in Section IV-D. Optimality here concerns estimated per-policy objectives within the candidate bank; transition effects and network conditions can affect the realized overhead of the complete flow. The evaluation that follows measures the resulting changes in service classification, communication overhead, and application-data recovery.

TABLE II
EVALUATED TRAFFIC CATEGORIES

ID	Category	Applications
S0	Streaming	Vimeo, Netflix, YouTube
S1	VoIP	Zoiper
S2	Chat	Skype
S3	Command & Control	SSH, RDP
S4	File Transfer	SFTP, RSYNC, SCP

TABLE III
FLOWSHIFT FILE-TRANSFER EMULATION RESULTS

Flow	Size (MB)	Bytes (%)	FCT (%)
F1	5	11.54	12.85
F2	25	11.64	12.89
F3	50	11.63	12.86
F4	100	11.64	12.86

VI. EVALUATION

The evaluation addresses two questions: whether FlowShift reduces the accuracy of encrypted-service classification, and whether it preserves application data during live transmission. We first conduct a dataset-based study under VPN and non-VPN conditions and then validate the sender–receiver pipeline in Mininet.

A. Dataset and Preprocessing

We use the MIT Lincoln Laboratory VPN/Non-VPN Network Application Traffic (VNAT) dataset [1]. The evaluation covers the five service categories shown in Table II. Each packet capture is converted into bidirectional flow records with CICFlowMeter [2], which supplies the statistical features used by the classifier and by the offline profile-construction stage.

Profiles for packet size, IAT, and volume are derived from representative subsets of each service class. Source flows are then transformed toward a different class profile, after which the same feature-extraction procedure is applied to the resulting traces. This produces directly comparable original and transformed flow representations.

B. Adversarial Classifier and Metrics

We use a Random Forest classifier as the traffic-analysis adversary. The data are divided using an 80/20 stratified train–test split to preserve class proportions. The classifier contains 100 trees, uses a maximum depth of 20, and applies class balancing. It is trained on original flows and evaluated on both original and transformed test flows.

Privacy effectiveness is measured by the decrease in true-service classification accuracy and by the redistribution of predictions away from the diagonal of the confusion matrix. Communication cost is reported using bandwidth overhead

$$O_B = \frac{B_{\text{shift}} - B_{\text{original}}}{B_{\text{original}}} \times 100\%, \tag{22}$$

and flow-completion-time overhead

$$O_F = \frac{F_{\text{shift}} - F_{\text{original}}}{F_{\text{original}}} \times 100\%. \tag{23}$$

For the emulation, restoration correctness is additionally checked by comparing the transmitted and reconstructed files byte for byte.

C. Dataset-Based Results

Figures 5 and 6 summarize the non-VPN and VPN results. The baseline confusion matrices contain strong diagonals, showing that the five services are readily distinguishable before protection. After transformation, predictions move away from the source labels and toward alternative services. The same qualitative change appears in both environments, indicating that the transformation affects service-level metadata in addition to any signature introduced by VPN encapsulation.

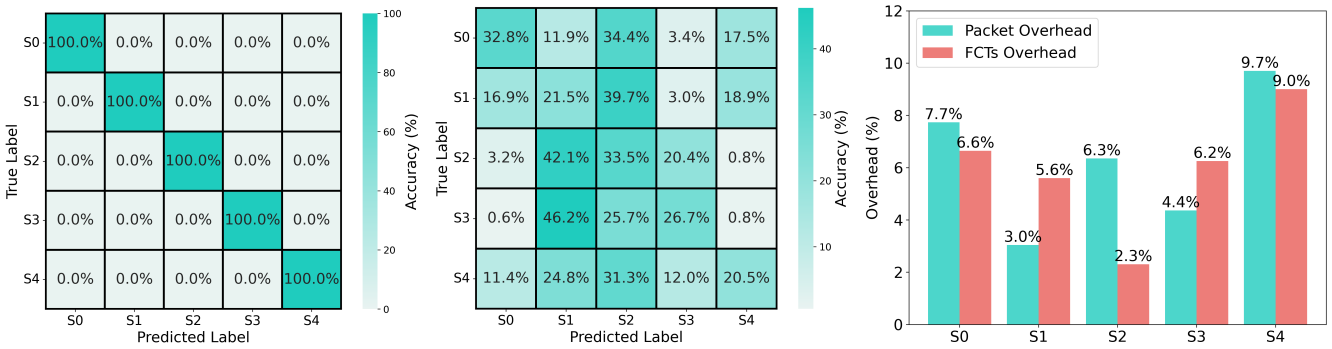
The transformation reduces classification accuracy by as much as 77% in the evaluated configurations. This reduction should be interpreted together with the confusion matrices: the objective is not simply to produce an error, but to shift observations toward plausible non-source classes. The reported average bandwidth overhead is 4.35% for non-VPN traffic and 3.144% for VPN traffic. Average FCT increases by 6.16% and 6.5%, respectively. These values reflect the combined cost of padding, dummy traffic, buffering, and pacing; the contribution of each mechanism depends on the distance between the selected source and target profiles.

D. Mininet Testbed

We implement the FlowShift encoder and decoder in Python 3.10 and deploy them on two Mininet hosts connected through Open vSwitch, as shown in Fig. 7. The testbed runs on Ubuntu 24.04 LTS with an Intel Core i9-14900K processor, 128 GB of DDR5 memory, and a 2 TB NVMe SSD. The emulated link capacity is 250 Mbps. Traffic is captured at the network interface to compute transmitted bytes and completion time, while hashes of the source and reconstructed files are compared to verify correctness.

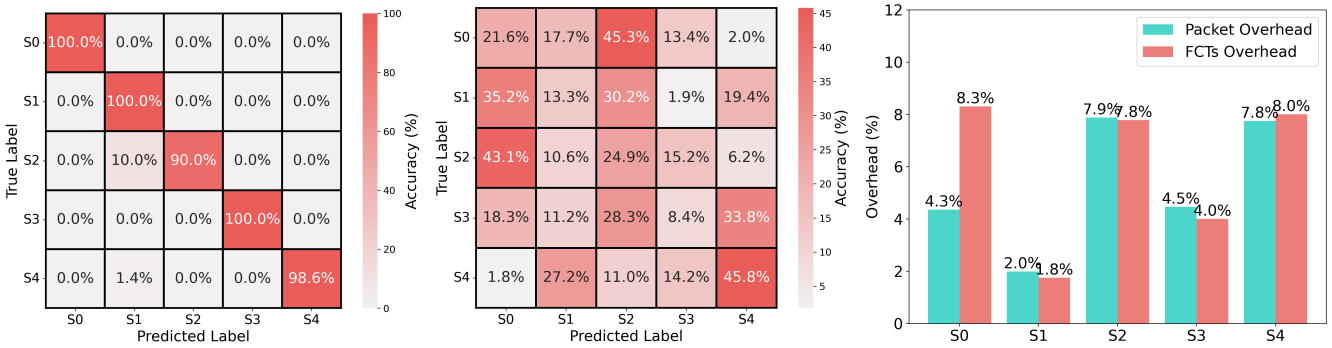
File transfer is used as the workload because it combines large payloads with sensitivity to both throughput and transformation correctness. Four files ranging from 5 to 100 MB are transmitted through the complete encoder–decoder path. Table III shows that bandwidth overhead remains between 11.54% and 11.64%, while FCT overhead remains between 12.85% and 12.89%. The narrow range across file sizes suggests that the per-epoch policy scales predictably with flow volume.

All four files are reconstructed exactly at the receiving endpoint. No flow reset or application-visible corruption is observed. The live traces are also predominantly assigned to non-file-transfer services by the evaluated classifier, showing that the statistical shift persists when transformation and



(a) Original traffic (b) Transformed traffic (c) Overhead by service

Fig. 5. FlowShift results for non-VPN traffic: classifier behavior before and after transformation, followed by bandwidth and FCT overhead.



(a) Original traffic (b) Transformed traffic (c) Overhead by service

Fig. 6. FlowShift results for VPN traffic: classifier behavior before and after transformation, followed by bandwidth and FCT overhead.

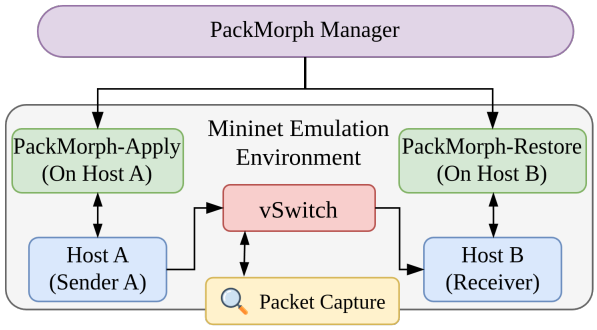


Fig. 7. Mininet topology used for end-to-end FlowShift validation.

restoration are executed in real time. The emulation overhead is higher than the dataset-level average because this experiment exercises the complete tunnel, including synchronization metadata and a sustained file-transfer workload.

VII. Discussion

A. Interpretation and Privacy–Performance Trade-off

The accuracy reduction and off-diagonal movement in Figs. 5 and 6 indicate that jointly modifying packet size, timing, and volume shifts flows away from their original service representations. This result supports FlowShift's objective for the evaluated Random Forest and feature set, but does not imply indistinguishability against every classifier. The Mininet experiment further shows that this shift can coexist with exact application-data recovery.

Greater displacement can require additional padding, artificial traffic, or buffering, thereby increasing bandwidth or FCT. The multi-objective selection in Section V captures this trade-off by retaining feasible, non-dominated policies. The higher testbed overhead reflects the complete sender–receiver path, including synchronization and sustained transfer behavior. Policy holding time introduces a related trade-off: shorter intervals permit more frequent changes but increase transition activity, whereas longer intervals reduce that activity but may retain an unsuitable policy and accumulate buffering.

B. Security, Limitations, and Deployment Implications

Synchronization state and restoration metadata must be authenticated, bound to the correct flow and interval, and protected against replay. Their representation should also avoid creating a stable signature that identifies FlowShift traffic. The framework reshapes selected metadata features but does not conceal endpoint addresses, routes, start times, or all long-term volume information.

The evaluation is limited to a flow-level Random Forest and a controlled file-transfer testbed. Defense-aware sequence models may exploit residual temporal patterns, while congestion, loss, retransmissions, and jitter may reduce target-profile fidelity. Future work should therefore study raw-sequence adversaries, capture-disjoint evaluation, interactive and lossy networks, and adaptive profiles and policy intervals driven by current queue and path conditions.

VIII. Conclusion

FlowShift addresses metadata leakage by changing the statistical identity presented by an encrypted flow rather than altering its protected payload. The framework derives service profiles offline, translates a selected profile into packet-size, timing, and volume actions, and coordinates the sender and receiver so that every artificial modification can be reversed. Evaluation on VPN and non-VPN VNAT traffic shows substantial degradation of service classification with modest average overhead, while Mininet experiments demonstrate exact restoration for file transfers up to 100 MB. The results support cooperative, target-driven transformation as a practical direction for reducing encrypted-traffic inference. They also identify the central challenges for deployment: secure synchronization, adaptive target selection, robustness to network dynamics, and evaluation against defense-aware sequence classifiers.

References

[1] S. Jorgensen, J. Holodnak, J. Dempsey, K. de Souza, A. Raghunath, V. Rivet, N. DeMoes, A. Alejos, and A. Wollaber, "Extensible machine learning for encrypted network traffic application labeling via uncertainty quantification," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 1, pp. 420–433, 2023.

[2] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani *et al.*, "Toward generating a new intrusion detection dataset and intrusion traffic characterization." *ICISSp*, vol. 1, no. 2018, pp. 108–116, 2018.